

Hashing Revision

12 problems

Fast recall cards for the current hashing package. Use Quick Review first, then expand a row when you need the full explanation. Exported in expanded card mode.

#01 FirstNonRepeating

Easy

Count first, preserve order second Easy [Source](#) [LeetCode](#)

Before reading: why is counting alone not enough to know the first unique character?

PROBLEM DESCRIPTION

Given a string, return the index of the first character that appears exactly once. If every character repeats, return -1.

INPUT

```
s = "loveleetcode"
```

OUTPUT

2

TIME

 $O(n)$

SPACE

 $O(k)$, where k is the number of distinct characters

DATA STRUCTURES

HashMap<Character, Integer>
frequency count

WHEN TO RECOGNIZE IT

When the answer depends on both frequency and original order: count all characters first, then scan the string again to find the earliest count-one character.

VISUAL HOOK

l:1

o:2

v:1

e:4

scan left to right

first count 1 = v

index 2

- 1 First pass records how often each character appears.
- 2 Second pass walks the original string order.
- 3 The first character with count one gives the answer index.

Picture the map as a frequency board, not the final answer. The second pass is what preserves the left-to-right order of the original string.

CORE MOVE

Build a frequency map in one pass. Scan the string again from left to right and return the first index whose character has count one. If none exists, return -1.

CODE SKELETON

```
count = map
for c in s:
    count[c]++
for i from 0 to s.length - 1:
    if count[s[i]] == 1:
        return i
return -1
```

BRUTE FORCE / BASELINE

For every character, scan the whole string to see whether it appears again. This preserves order but costs $O(n^2)$.

ALTERNATE APPROACHES

- Use an `int[26]` array instead of a `HashMap` when the input is guaranteed to be lowercase English letters.
- Use a `LinkedHashMap` storing counts and first indexes, then inspect entries in insertion order; this still needs the full count information before the answer is reliable.

BUG MAGNETS

- A single pass cannot know that the current character will stay unique later.
- The map finds counts; the second scan finds the first index.
- If the character set is fixed and small, an array is simpler and faster than a `HashMap`.

#02 LargestUniqueNumber

Easy

Count first, choose largest count-one value Easy [Source](#) [LeetCode](#)

Before reading: why must you finish counting before trusting a number as unique?

PROBLEM DESCRIPTION

Given an integer array, return the largest number that appears exactly once. If every number repeats, return -1.

INPUT

nums = [5, 7, 3, 9, 4, 9, 8, 3, 1]

OUTPUT

8

TIME
 $O(n)$

SPACE
 $O(k)$, where k is the number of distinct values

DATA STRUCTURES
`HashMap<Integer, Integer>`
frequency count

WHEN TO RECOGNIZE IT

When uniqueness is based on frequency, but the answer asks for the largest unique value instead of the first position.

VISUAL HOOK

5:1

7:1

3:2

9:2

4:1

8:1

1:1

max = 8

- 1 Count how many times every number appears.
- 2 Ignore every number whose count is not one.
- 3 Among the count-one numbers, keep the largest value.

Picture the map as a scoreboard. Cross out every value whose count is more than one, then pick the biggest value still standing.

CORE MOVE

Count every number first. Scan the array values and keep the maximum number whose count is exactly one. If no such value exists, return -1.

CODE SKELETON

```
count = map
for num in nums:
    count[num]++
answer = -1
for num in nums:
    if count[num] == 1:
        answer = max(answer, num)
return answer
```

BRUTE FORCE / BASELINE

For each number, scan the whole array to count how often it appears, then keep the largest number with count one. It is correct but costs $O(n^2)$.

ALTERNATE APPROACHES

- Use a TreeMap or sorted keys after counting, then scan from largest to smallest until a count-one value appears; this is cleaner for ordered lookup but costs extra sorting/tree overhead.
- If the value range is small and fixed, use a frequency array instead of a HashMap.

BUG MAGNETS

- Do not update the answer while counting; a value that looks unique early may repeat later.
- Return -1 when no count-one value exists.
- The second pass can scan the original array or the map entries; both are fine for $O(n)$.

#03 MaximumNumberOfBalloons

Easy

Word frequency limit Easy [Source](#) [LeetCode](#)

Before reading: which letter in "balloon" is the bottleneck?

PROBLEM DESCRIPTION

Given a text string, return how many times you can form the word "balloon" using each character from the text at most once.

INPUT

text = "loonbalxballpoon"

OUTPUT

2

TIME

$O(n)$

SPACE

$O(1)$, only lowercase letter counts

DATA STRUCTURES

HashMap<Character, Integer> or int[26] frequency count

WHEN TO RECOGNIZE IT

When a target word has repeated letters and the answer is limited by the scarcest required character.

VISUAL HOOK

b:2

a:2

l:4

o:4

n:2

b/1=2

a/1=2

l/2=2

o/2=2

n/1=2

- 1 Count letters in the text.
- 2 Compare only b, a, l, o, and n.
- 3 Divide l and o by two, then take the minimum.

Picture each required letter as a bucket. Because "balloon" needs two l's and two o's, divide those counts by two before taking the minimum bucket.

CORE MOVE

Count letters in the text. For each character needed by "balloon", divide available count by required count. The smallest quotient is the number of full words you can build.

CODE SKELETON

```
count = letter counts from text
return min(
    count['b'],
    count['a'],
    count['l'] / 2,
    count['o'] / 2,
    count['n']
)
```

BRUTE FORCE / BASELINE

Repeatedly try to remove the letters b, a, l, l, o, o, n from the text until one removal fails. It works, but repeated searching/removal is clumsy and can become $O(n \times \text{answer})$.

ALTERNATE APPROACHES

- Use an int[26] array and directly compute $\min(\text{count}[b], \text{count}[a], \text{count}[l] / 2, \text{count}[o] / 2, \text{count}[n])$. This is the cleanest version for lowercase English letters.
- Build a target frequency map for a generic word; slightly more general, but more code than needed for fixed "balloon".

BUG MAGNETS

- Remember l and o are required twice.
- The answer is the minimum quotient across required letters, not the total count of matching letters.
- Extra unrelated letters in text do not matter.

#04 LongestPalindrome

Easy

Pair counts plus one center Easy [Source](#) [LeetCode](#)**Before reading: how many odd-count characters can stand in the middle?****PROBLEM DESCRIPTION**

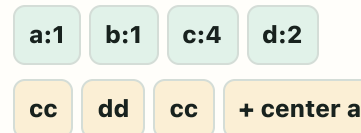
Given a string, return the length of the longest palindrome that can be built using its letters. Characters are case-sensitive.

INPUT`s = "abcccd"`**OUTPUT**

7

TIME **$O(n)$** **SPACE** **$O(k)$, where k is the number of distinct characters****DATA STRUCTURES****HashMap<Character, Integer>
frequency count****WHEN TO RECOGNIZE IT**

When order does not matter and you only need the longest palindrome length: every pair can be used, and at most one odd leftover can sit in the middle.

VISUAL HOOK

- 1 Count every character.
- 2 Use all possible pairs from each count.
- 3 Add one extra center only if some character has an unused odd leftover.

Picture building the palindrome from mirrored pairs first. After all pairs are placed, one leftover single character may become the center.

CORE MOVE

Count character frequencies. Add the largest even portion from every count. If any character had an unused odd leftover, add one more for the center.

CODE SKELETON

```
count = map
length = 0
hasOddLeftover = false
for freq in count values:
    length += (freq / 2) * 2
    if freq is odd:
        hasOddLeftover = true
if hasOddLeftover:
    length++
return length
```

BRUTE FORCE / BASELINE

Try building palindromes by choosing letters and checking whether the arrangement can mirror. That wastes effort because only counts matter.

ALTERNATE APPROACHES

- Track a Set of unmatched characters: add when first seen, remove when a pair is completed, and increase length by two for every completed pair. Add one center if the set is non-empty.
- Use an int array when the character range is known; use a HashMap for general characters.

BUG MAGNETS

- Only one odd leftover can be used as the center.
- Do not add every odd count fully; add count - 1 as pairs, then maybe one center.
- Uppercase and lowercase are different characters on LeetCode.

#05 RansomNote

Easy

Spend magazine letters Easy [Source](#) [LeetCode](#)

Before reading: which string is the supply and which string spends letters?

PROBLEM DESCRIPTION

Given two strings, return true if the ransom note can be built using letters from the magazine, where each magazine character can be used only once.

INPUT

ransomNote = "hello", magazine = "helloworld"

OUTPUT

true

TIME **$O(n + m)$** **SPACE** **$O(1)$, 26 lowercase letter counts****DATA STRUCTURES****int[26] character bank****WHEN TO RECOGNIZE IT**

When one string is a supply bank and the other string spends from that bank one character at a time.

VISUAL HOOK**h:1****e:1****l:3****o:2****spend h****spend e****spend l****spend l****spend o**

- 1 Count magazine letters as available supply.
- 2 Spend one count for each ransom note letter.
- 3 If any letter balance goes negative, return false.

Picture magazine as a bank of letters. The note withdraws letters; if any balance goes negative, construction is impossible.

CORE MOVE

Count all magazine letters. For each ransom note letter, decrement its count. If any count becomes negative, return false; otherwise return true.

CODE SKELETON

```
bank = int[26]
for c in magazine:
    bank[c]++
for c in ransomNote:
    bank[c]--
    if bank[c] < 0:
        return false
return true
```

BRUTE FORCE / BASELINE

For every note character, search the magazine for an unused matching character and mark it consumed. This is straightforward but can take $O(n \times m)$.

ALTERNATE APPROACHES

- Use a HashMap if the character set is not fixed to lowercase English letters.
- Return false immediately during the note scan when a count drops below zero; the final scan over all counts is then unnecessary.

BUG MAGNETS

- Each magazine character can be used only once.
- Count magazine first, not ransom note first, because magazine is the supply.
- With lowercase letters, int[26] is simpler than a HashMap.

#06 TwoSum

EasyStore complement while scanning Easy [Source](#) [LeetCode](#)

Before reading: what partner value is the current number looking for?**PROBLEM DESCRIPTION**

Given an integer array and a target, return the indexes of two different numbers whose values add up to the target. Exactly one valid answer is guaranteed.

INPUT

nums = [3, 2, 4], target = 6

OUTPUT

[2, 1]

TIME

$O(n)$

SPACE

$O(n)$

DATA STRUCTURES

HashMap<Integer, Integer>
complement to index

WHEN TO RECOGNIZE IT

When each number needs a partner value: target - current. A map lets you ask whether this current number was needed by something earlier.

VISUAL HOOK

3 needs 3

2 needs 4

4 found

bank: 3->0

bank: 4->1

answer: 2,1

- 1 Scan one number at a time.
- 2 If the current number is already in the complement bank, return both indexes.
- 3 Otherwise store the value needed to pair with this number.

Picture the map as a wish list. Each earlier number stores the value it is waiting for; when the current number appears in the wish list, the pair is complete.

CORE MOVE

Scan left to right. If the current number already exists in the complement map, return current index plus the stored index. Otherwise store target - current as a needed future value.

CODE SKELETON

```
needToIndex = map
for i from 0 to nums.length - 1:
    if nums[i] in needToIndex:
        return [i, needToIndex[nums[i]]]
    needToIndex[target - nums[i]] = i
```

BRUTE FORCE / BASELINE

Try every pair i, j and check whether $\text{nums}[i] + \text{nums}[j]$ equals the target. This is simple but costs $O(n^2)$.

ALTERNATE APPROACHES

- Store value to index instead of complement to index; then check whether target - current already exists before adding current.
- Sorting plus two pointers works for values, but it loses original indexes unless you carry pairs, so the hash map is cleaner.

BUG MAGNETS

- Check the map before inserting the current number so the same element is not reused.
- Duplicate values are valid if they are at different indexes.
- Returning indexes in either order is accepted.

#07 GroupAnagrams

Medium

Same signature means same group Medium [Source](#) [LeetCode](#)**Before reading: what key makes two rearranged words look identical?****PROBLEM DESCRIPTION**

Given an array of strings, group together the words that are anagrams of each other. The groups and words can be returned in any order.

INPUT

```
strs = ["eat", "tea", "tan", "ate", "nat", "bat"]
```

OUTPUT

```
[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]
```

TIME **$O(n * k)$** **SPACE** **$O(n * k)$** **DATA STRUCTURES****HashMap<String,
List<String>>, int[26]
frequency signature****WHEN TO RECOGNIZE IT**

When words are equivalent after rearranging letters: convert each word into a stable signature, then use that signature as the group key.

VISUAL HOOK

eat -> aet

tea -> aet

ate -> aet

tan -> ant

nat -> ant

bat -> abt

- 1 Build the same canonical key for all anagrams.
- 2 Use that key as the HashMap bucket.
- 3 Append the original word to that bucket.

Picture every word wearing a sorted-letter name tag. Words with the same name tag walk into the same bucket.

CORE MOVE

For each string, build a canonical key from its letter counts. Put the original string into the list stored at that key. Return all map values.

CODE SKELETON

```
groups = map key -> list
for word in strs:
    key = letter-count signature(word)
    groups[key].add(word)
return all group lists
```

BRUTE FORCE / BASELINE

Compare every pair of strings with an anagram check and build groups manually. It works but can take $O(n^2 * k)$.

ALTERNATE APPROACHES

- Sort each string and use the sorted string as the key; simpler to write, but costs $O(n * k \log k)$.
- Use a delimiter-based count key such as `#1#0#0...` to avoid ambiguity when counts have multiple digits.

BUG MAGNETS

- Use the original string in the group, not the sorted/signature string.
- The signature must be unambiguous; raw count concatenation can collide without delimiters or fixed reconstruction.
- Empty string is a valid key and should form its own group.

#08 IsomorphicStrings

Easy

Two-way character mapping Easy [Source](#) [LeetCode](#)

Before reading: why is one map not enough for badc -> baba?

PROBLEM DESCRIPTION

Given two strings of the same length, return true if characters in the first string can be consistently replaced to form the second string. Each source character maps to one target character, and no two source characters may map to the same target character.

INPUT

`s = "badc", t = "baba"`

OUTPUT

false

TIME

$O(n)$

SPACE

$O(k)$, where k is the number of distinct characters

DATA STRUCTURES

Two `HashMaps<Character, Character>` or two int arrays

WHEN TO RECOGNIZE IT

When the strings must keep the same position pattern: repeated characters in one string must line up with repeated characters in the other.

VISUAL HOOK**b->b****a->a****d->b breaks****target b already taken****need one-to-one****false**

- 1 Read both strings at the same index.
- 2 Check s-to-t mapping for consistency.
- 3 Check t-to-s mapping so no target character is reused by a different source.

Picture every character wearing a reserved partner tag. The forward map checks consistency from s to t; the reverse map prevents two different s characters from sharing one t character.

CORE MOVE

Scan both strings together. For each pair, verify the old mapping in both directions if it exists. If either direction conflicts, return false; otherwise store both mappings.

CODE SKELETON

```
forward = map
reverse = map
for i from 0 to n - 1:
    a = s[i]
    b = t[i]
    if forward[a] exists and forward[a] != b:
        return false
    if reverse[b] exists and reverse[b] != a:
        return false
    forward[a] = b
    reverse[b] = a
return true
```

BRUTE FORCE / BASELINE

For every pair of positions i and j, check whether $s[i] == s[j]$ has the same truth value as $t[i] == t[j]$. This checks the pattern directly but costs $O(n^2)$.

ALTERNATE APPROACHES

- Use two `int[256]` arrays storing last seen positions. If the last seen positions differ at any index, the pattern differs.
- Encode each string into a first-seen pattern such as 1,2,2 and compare the encoded patterns.

BUG MAGNETS

- Checking only $s \rightarrow t$ is not enough; two source characters could map to the same target.
- A character may map to itself, but still only one source character can own that target character.
- The order of characters means positions stay fixed; replacements do not rearrange the string.

#09 WordPattern

Easy

Pattern char to word bijection Easy [Source](#) [LeetCode](#)

Before reading: why must dog be owned by only one pattern letter?

PROBLEM DESCRIPTION

Given a pattern string and a sentence, return true if each pattern character maps consistently to exactly one word, and no two pattern characters map to the same word.

INPUT

pattern = "abba", s = "dog cat cat dog"

OUTPUT

true

TIME
 $O(n)$

SPACE
 $O(k)$, where k is the number of distinct pattern characters and words

DATA STRUCTURES
Two HashMaps<String, String> for pattern-to-word and word-to-pattern

WHEN TO RECOGNIZE IT

When positions in a short pattern must match whole words in a sentence one-to-one.

VISUAL HOOK

a->dog

b->cat

b->cat

a->dog

same pattern

same word

two-way lock

- 1 Split the sentence into words.
- 2 Check pattern-to-word consistency.
- 3 Check word-to-pattern consistency so two letters cannot share a word.

Picture each pattern letter reserving a word, and each word reserving a pattern letter. Both reservations must agree at every position.

CORE MOVE

Split the sentence into words. If the number of words differs from the pattern length, return false. Walk both arrays together and verify both maps before storing the current pair.

CODE SKELETON

```
words = s.split spaces
if pattern.length != words.length: return false
forward = map
reverse = map
for i from 0 to words.length - 1:
    p = pattern[i]
    w = words[i]
    if forward[p] exists and forward[p] != w:
return false
    if reverse[w] exists and reverse[w] != p:
return false
    forward[p] = w
    reverse[w] = p
return true
```

BRUTE FORCE / BASELINE

For every pair of positions, compare whether `pattern[i] == pattern[j]` matches `words[i].equals(words[j])`. This directly checks the pattern but costs $O(n^2)$.

ALTERNATE APPROACHES

- Use a single map plus a HashSet of already-used words to block two pattern letters from sharing one word.
- Encode pattern and words into first-seen IDs, then compare the two ID sequences.

BUG MAGNETS

- Check word count against pattern length before mapping.
- One-way mapping is not enough; two pattern letters cannot share one word.
- Split on spaces carefully; the LeetCode input uses single spaces between words.

#10 MyHashMap

Easy

Bucket array with collision chain Easy [Source](#) [LeetCode](#)

Before reading: after hashing to a bucket, where can the key actually be?

PROBLEM DESCRIPTION

Design a hash map with put, get, and remove operations without using a built-in hash table. Each key maps to one value, get returns -1 for missing keys, and put updates an existing key.

INPUT

`put(1,10), put(1001,30), put(2001,40),
remove(1001), get(2001)`

OUTPUT

40

TIME

Average $O(1)$, worst $O(\text{chain length})$

SPACE

$O(\text{bucket count} + \text{stored keys})$

DATA STRUCTURES

Node[] buckets, doubly linked collision chains

WHEN TO RECOGNIZE IT

When the problem asks you to build the table mechanics yourself: bucket index first, then walk only that bucket's chain.

VISUAL HOOK

bucket 1

2001:40

1001:30

1:10

remove 1001

link 2001 -> 1

get 2001 = 40

- 1 Compute `key % bucketCount` to choose the bucket.
- 2 Search only that bucket's linked chain.
- 3 When removing, reconnect prev and next and fix the bucket head if needed.

Picture the array cell as a parking slot. Colliding keys park in a linked line at that slot; remove must reconnect the cars before and after the removed key.

CORE MOVE

Hash the key to a bucket. Put inserts or updates inside that bucket's chain. Get searches that chain. Remove unlinks the matching node and updates the bucket head when needed.

CODE SKELETON

```
bucket = key % bucketCount
put:
    find key in bucket chain
    update if found, else insert node
get:
    find key in bucket chain
    return value or -1
remove:
    find key in bucket chain
    unlink node; if head, move bucket head
```

BRUTE FORCE / BASELINE

Store key-value pairs in one list and scan the whole list for every operation. It is easy to write but every put, get, and remove can become $O(n)$.

ALTERNATE APPROACHES

- Use a larger bucket array or resize when chains get long to keep average operations closer to $O(1)$.
- Use singly linked chains instead of doubly linked nodes, but removal needs tracking the previous node while scanning.

BUG MAGNETS

- Updating an existing key must change the value, not insert a duplicate node.
- Removing the head of a chain must update the bucket array cell.
- Collision tests need keys with the same $\text{key} \% \text{bucketCount}$, such as 1, 1001, and 2001.

#11 LongestConsecutiveSequence

Medium

Start only at chain heads Medium [Source](#) [LeetCode](#)

Before reading: why should counting start only when num - 1 is missing?

PROBLEM DESCRIPTION

Given an unsorted integer array, return the length of the longest run of consecutive values. The algorithm must run in $O(n)$ time.

INPUT

nums = [100, 4, 200, 1, 3, 2]

OUTPUT

4

TIME
 $O(n)$

SPACE
 $O(n)$

DATA STRUCTURES
HashSet<Integer> for membership, optional visited set

WHEN TO RECOGNIZE IT

When order in the array does not matter, but consecutive numeric neighbors matter. Use the set for $O(1)$ membership and only expand from true starts.

VISUAL HOOK



- 1 Put every number into a HashSet.
- 2 Skip any number that has a left neighbor.
- 3 From a true start, walk right while the next number exists.

Picture each number as a dot on a number line. Only start walking from a dot that has no left neighbor; that avoids recounting the same chain from the middle.

CORE MOVE

Put every number in a set. For each number, if $\text{num} - 1$ is not in the set, treat it as the left edge of a sequence and walk upward while $\text{num} + \text{offset}$ exists.

CODE SKELETON

```
set = all nums
best = 0
for num in nums:
    if set does not contain num - 1:
        length = 1
        while set contains num + length:
            length++
        best = max(best, length)
return best
```

BRUTE FORCE / BASELINE

Sort the array, then scan and count consecutive runs. It is easy and handles duplicates, but sorting costs $O(n \log n)$, which misses the required $O(n)$.

ALTERNATE APPROACHES

- Use a HashMap boundary-length merge approach that updates sequence lengths at the ends; it is $O(n)$, but harder to remember.
- A visited set can avoid repeated duplicate work, but the key optimization is still starting only where $\text{num} - 1$ is absent.

BUG MAGNETS

- Do not start counting from every number; that can become $O(n^2)$.
- Duplicates should not reset or inflate a sequence.
- An empty input should return 0.

#12 TopKFrequentElements

Medium

Frequency buckets from high to low Medium [Source](#) [LeetCode](#)

Before reading: after counting values, what should be sorted by frequency without sorting?

PROBLEM DESCRIPTION

Given an integer array and an integer k, return the k values that appear most frequently. The answer can be returned in any order.

INPUT

nums = [1, 2, 1, 2, 1, 2, 3, 1, 3, 2], k = 2

OUTPUT

[1, 2]

TIME

$O(n)$

SPACE

$O(n)$

DATA STRUCTURES

HashMap<Integer, Integer>
counts, frequency-to-values
buckets

WHEN TO RECOGNIZE IT

When the answer asks for most frequent values and k can be smaller than the number of distinct values.

VISUAL HOOK

1:4

2:4

3:2

freq 4 -> [1,2]

freq 2 -> [3]

take first k

- 1 Count each number's frequency.
- 2 Put each number on the bucket for its count.
- 3 Walk buckets from highest frequency down until k values are collected.

Picture numbers first going into count boxes, then those counts becoming shelves. Walk shelves from highest frequency downward until k values are collected.

CORE MOVE

Count each value. Group values by their frequency. Iterate frequencies from nums.length down to 1 and collect values until k slots are filled.

CODE SKELETON

```
freq = map value -> count
buckets = map count -> values
for value, count in freq:
    buckets[count].add(value)
answer = []
for count from nums.length down to 1:
    for value in buckets[count]:
        add value until answer size == k
return answer
```


BRUTE FORCE / BASELINE

Count frequencies, then repeatedly scan all distinct values to find the current largest remaining count. This works but can take $O(k * \text{distinct})$.

ALTERNATE APPROACHES

- Use a min-heap of size k over the frequency map; this is $O(n \log k)$ and useful when bucket memory is not preferred.
- Sort distinct values by frequency; simpler to reason about, but $O(d \log d)$ where d is distinct count.

BUG MAGNETS

- Bucket index is frequency, not the value itself.
- Multiple values can share the same frequency.
- Stop after k values; do not keep adding an entire bucket if it would exceed k .